DEEP LEARNING: IDENTIFYING THE SOUND OF SEATTLE BIRDS

Abstract:

This study uses deep learning to define, compile and train CNN models to identify the sound of the birds found in the city of Seattle. We use neural networks and convoluted neural networks to train our models. The sound of birds in this study is from twelve birds found in Seattle, namely, American Crow, American Robin, Bewick's Wren, Black-capped Chickadee, Dark-eyed Junco, House Finch, House Sparrow, Northern Flicker, Red-winged Blackbird, Song Sparrow, Spotted Towhee and White-crowned Sparrow.

Introduction:

The data comes from a Kaggle competition called **Xeno-Canto Bird Recordings Extended (A-M)**. The sounds of twelve birds from the city of Seattle were selected from this huge dataset. The list of birds is as follows: American Crow, American Robin, Bewick's Wren, Black-capped Chickadee, Dark-eyed Junco, House Finch, House Sparrow, Northern Flicker, Red-winged Blackbird, Song Sparrow, Spotted Towhee and White-crowned Sparrow. These mp3 files were then converted into spectrograms, for facilitating our neural networks. Images are much easier to feed to a model than an mp3 file.

Let us see how the conversion from an mp3 file to a spectrogram happens.

- The sound clip is subsampled to 22050 Hz.
- The first 3 seconds of sound clip are selected.
- A spectrogram is produced for each 2-second window, resulting in a 128 (frequency) x 517 (time) 'image' of the bird call.

The data was given in HDF5 format, which allows clean labeling and maintenance of the original tensor format.

Theoretical Background:

Deep Learning models are robust ML models that can differentiate between nonlinear and complex data. Unlike SVM and decision trees, they can differentiate between data that is multi-class as well as nonlinear. Though they are not as interpretable as the other models. Deep Learning is a much more complex technique of machine learning. It does not have any visual plots like SVMs or decision trees, to help us understand how the model comes to a specific output. Neural networks are a part of deep learning that have the foundation of back propagation.

Back propagation makes use of the gradient descent algorithms to update the weights of the network. We shall see what weights are in detail shortly. Back propagation is mainly used to train feed-forward networks.

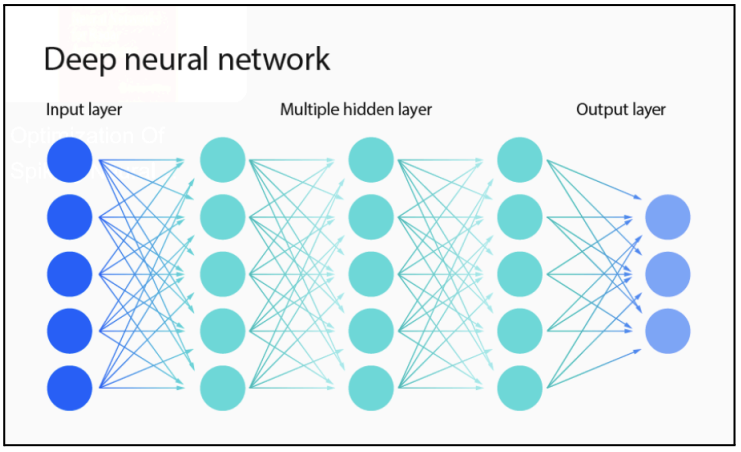


Figure 1: Visual Depiction of a Neural Network

This is a pictorial representation of how a neural network runs. There are three layers: the input layer, the hidden layer(s) and the output layer. We feed our input data to the input layer. And we receive our output at the output layer, as the names suggest. The hidden layer is a layer with activation functions. An activation function as the name suggests is a mathematical function. It helps the neural network compute or model the output. It also adds non-linearity to the model which helps us process complex data in neural networks. Different applications of neural networks call for the use of a different activation function, for example, a sigmoid activation function is mainly used for binary classification.

Each circle in the figure above is called a node. Each connection between two nodes has a weight 'w' attached to it. And every node has its own bias 'beta'. The weights plus biases together are called parameters. It is very common to end up with more parameters than data points or instances. This causes neural networks to overfit.

Overfitting is tackled by using 'dropout layers'. This layer basically helps you to set a percentage of data that can be ignored by the neural network. The weights and biases can be tuned according to the purpose of the neural network - linear or nonlinear transformations. Overfitting can also be overcome by EarlyStopping. It has a patience parameter, which tells the model how many epochs are we ready to wait for things or metrics in this case to change. The other parameter can be the threshold values of loss function, validation accuracy or some other metric. Below are the equations to calculate each layer of the neural network. f(X) is the output, A□ is the hidden layer equation and g(z) is the activation function, in this case a sigmoid.

$$f(X) = \beta_0 + \sum_{k=1}^{K} \beta_k A_k$$

$$A_k = g(w_{k0} + \sum_{j=1}^{p} w_{kj} X_j)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

Figure 2: Equations for Neural Network

An activation function allows us to select segments from our data and also helps us correct the format of our output.
We can put a bunch of neural networks together to get a desired output! They are extremely flexible to use and have a variety of applications, like image/document classification, convoluted neural networks (CNNs) and recurrent neural networks (RNNs).

Convoluted Neural Networks (CNNs):
These are commonly used for image classification. They have a set type and format of layers to them.

- Firstly, we slide a filter across an image and compute a smaller set of images.
- Output received by step 1 is a new, smaller image that highlights the areas where the original image matches the filter. This is the convoluted layer.
- The pooling layer condenses the image information into an even smaller summary image.

We can choose the number and size of convoluted filters for our CNN models. These filters are learnt by the model just like it learns weights and biases in real-time.

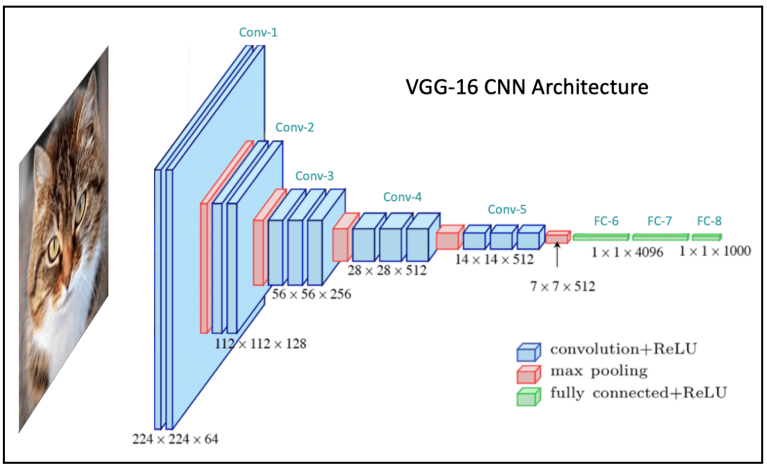


Figure 3: Architecture of CNNs

For Image Classification in particular, we can use data augmentation to help the learning process of our model. Data augmentation is just zooming in, out, mirroring or shifting of one image. This is also done when you do not have enough images to train your model. Using data augmentation you can expand your dataset.

Recurrent Neural Networks (RNNs):
These are neural networks that are used to process sequential data - data where order of every data point matters. Unlike traditional neural networks, RNNs have loops in their architecture, which allows them to persist and go on from one step to another. They remember past information and also handle variable length inputs. Most common applications of RNNs are NLP or Natural Language Processing, Time series Predictions or Video Analysis.

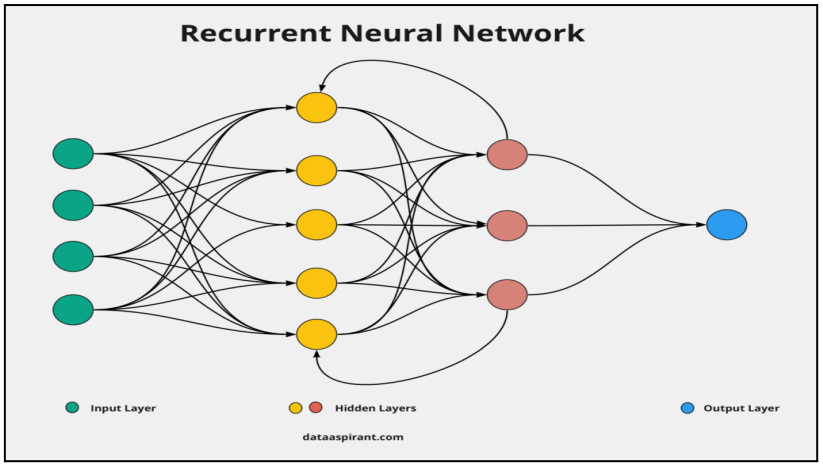


Figure 4: Architecture of RNNs

Disadvantages of RNNs:

- RNNs train very slowly as they have multiple loops to run. They are impossible to run parallely.
- Due to constant updates to the memory, RNNs have a hard time to retain information.
- LSTM or long short term memory keeps longer past information in memory, this is very expensive computationally.

Transformers:

They are currently replacing RNNs and CNNs. Transformers are better as they are used to learn 'context' of a given text, image, document, which make them very much more like humans than other neural networks. Transformers is what is used in generative AI models like ChatGPT and Gemini.

Methodology:

Data Preparation-

We used the dataset given to us in the HDF5 format, containing spectrograms of 12 species of birds. The spectrograms have a size of 128 x 512. First we plotted a bar graph of the number of samples per given species.

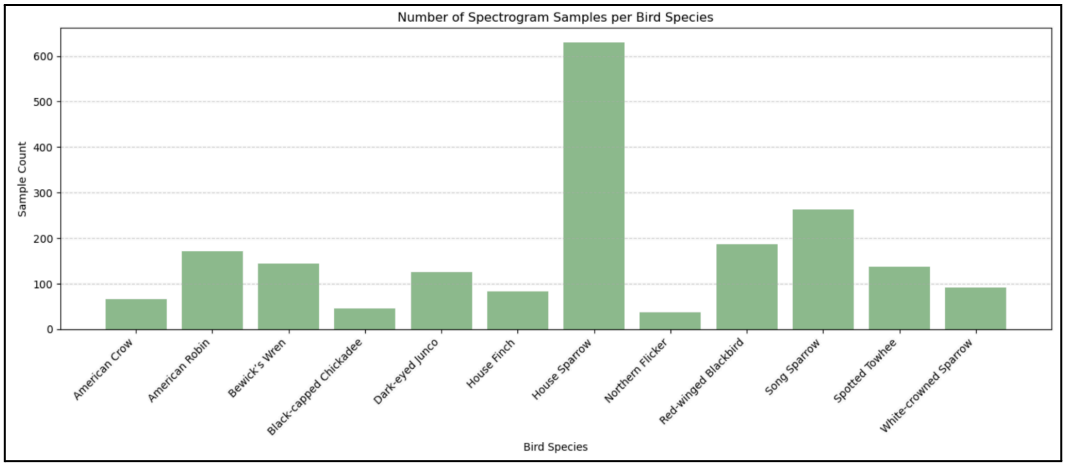


Figure 5: Plot of Samples vs Species

The number of samples per size are very nonuniform. We see that House Sparrow has the most samples. To avoid overfitting and a model biased to one particular bird species, we choose two species with similar numbers of samples for a binary classification using a neural network. Thus the chosen species were American Robin and Red-Winged Blackbird (~200 samples).

Data Normalisation-

To ensure consistent intensity scaling across samples, per-sample-min-max normalisation was used. This prevents dominant values in one sample from affecting model performance on others. The train to test split ratio was 75% and 25% respectively. Stratification was applied to maintain class balance.

Binary Model Architecture-

We used a CNN model for all the four binary classifiers. Their model architecture is as follows:

1. The first model is a simple basic model. It has two convolutional layers and also a dropout layer of 0.3. We then implement Early Stopping to see if it improves accuracy.
2. The second model has a more complex structure, with three convolutional layers and a dropout layer. We try to minimise overfitting by reducing the dense units in the final layer.
3. We have three convolutional layers in the third model, but unlike the previous one, these have bigger kernel sizes.
4. Our last model has two convolutional layers, but it uses AveragePooling instead of MaxPooling. AveragePooling preserves overall information of the model whereas MaxPooling highlights the strong features.

Multi-Class Model Architecture-

We continue with CNNs for multi-class models too. Their structure is as follows:

1. The first model is run on a subset of data as the data is too large. It has three convolutional layers, a basic model to start with.
2. The next model is on the full dataset, with three convolutional layers like the previous model but we increase the kernel size to fit the entire data. We then implement Early Stopping to improve accuracy.
3. We then model a more complex neural network. Instead of ReLU, we use LeakyReLU to handle the suspected dead neuron issues from the earlier models as the accuracy and overfitting issues still persist.
4. Our last model is more complex, and makes use of AveragePooling instead of MaxPooling and has broader kernel sizes.

Results:

Binary Model I Metrics :

Metric	Training Value	Validation Value
Accuracy	1.0000	0.7000
AUC	1.0000	0.8058
Loss	0.0092	0.7445
Precision	1.0000	0.6786
Recall	1.0000	0.8085

Table 1: Results for binary classifier I

The training values for metrics are 1.0 or close to 1.0, indicating a case of potential overfitting. The validation metrics are not perfect like the training metrics but they do not seem as bad either. But the table and the graphs tell us that the model trains very well but does not generalise well. The training lines are on the rise and much smoother than the validation curves which have a lot of fluctuations.

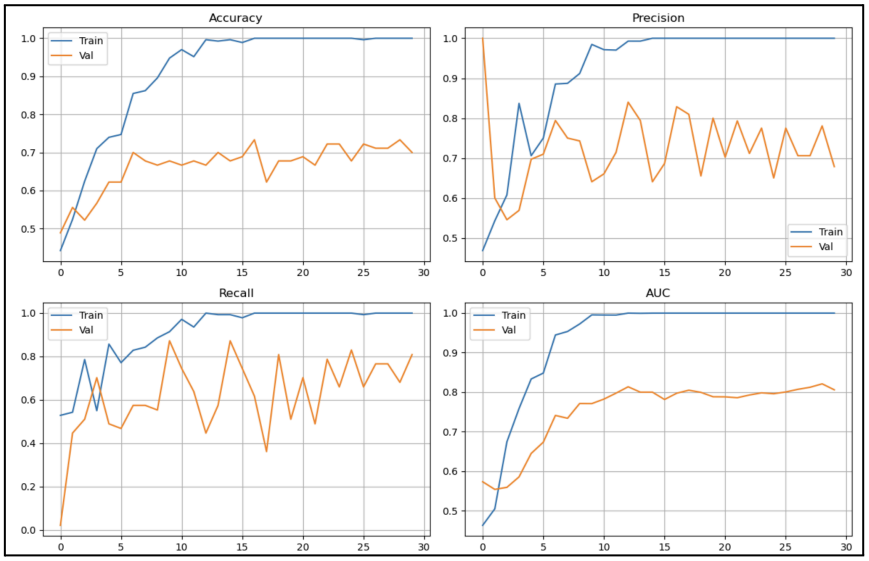


Figure 6: Plots - binary classifier I

Binary Model I Metrics with Early Stopping:

Metric	Training Value	Validation Value
Accuracy	0.9979	0.6556
AUC	1.0000	0.7566
Loss	0.0090	1.0070
Precision	0.9960	0.7353
Recall	1.0000	0.5319

Table 2: Results for binary classifier I (early stopping)

Since it is the same model, the training has been perfect. The testing metrics are still not great, and in the same range as the above model. The accuracy dips a little bit, and so does the AUC.

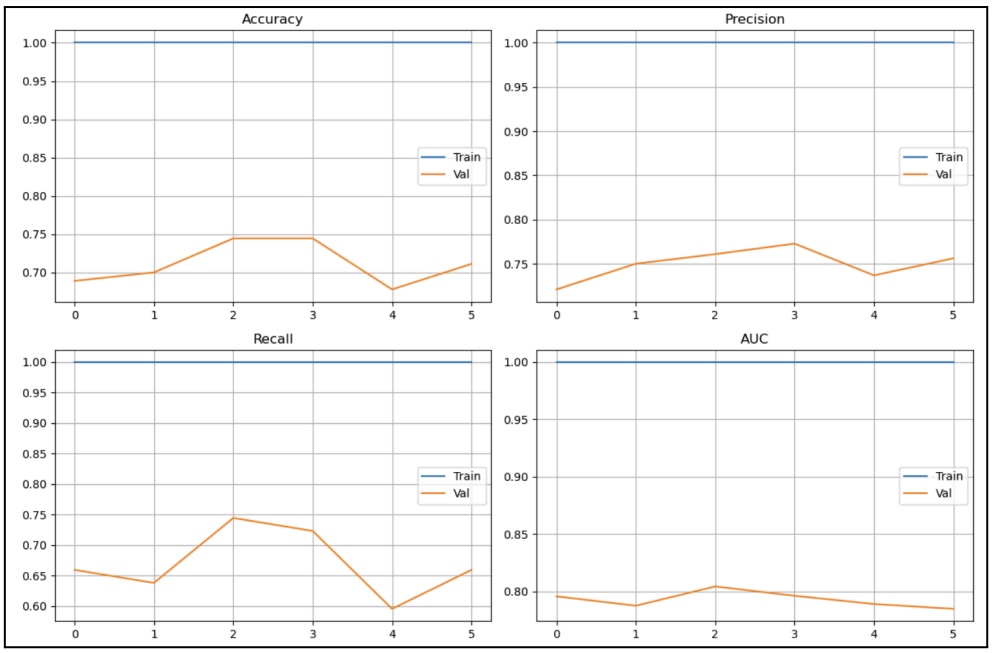


Figure 7: Plots - binary classifier I (early stopping)

Binary Model II Metrics:

Metric	Training Value	Validation Value
Accuracy	0.9931	0.7444
AUC	0.9999	0.8196
Loss	0.0087	0.8503
Precision	0.9922	0.6875
Recall	0.9949	0.9362

Table 3: Results for binary classifier II

The training metrics tell us that the model has trained well on the data with 99.3% accuracy, and an AUC pretty close to 1.0. But it struggles to generalise a little, as it has a test accuracy of 74.4% and an AUC of 0.81. The graph still has fluctuations for all the validation curves.

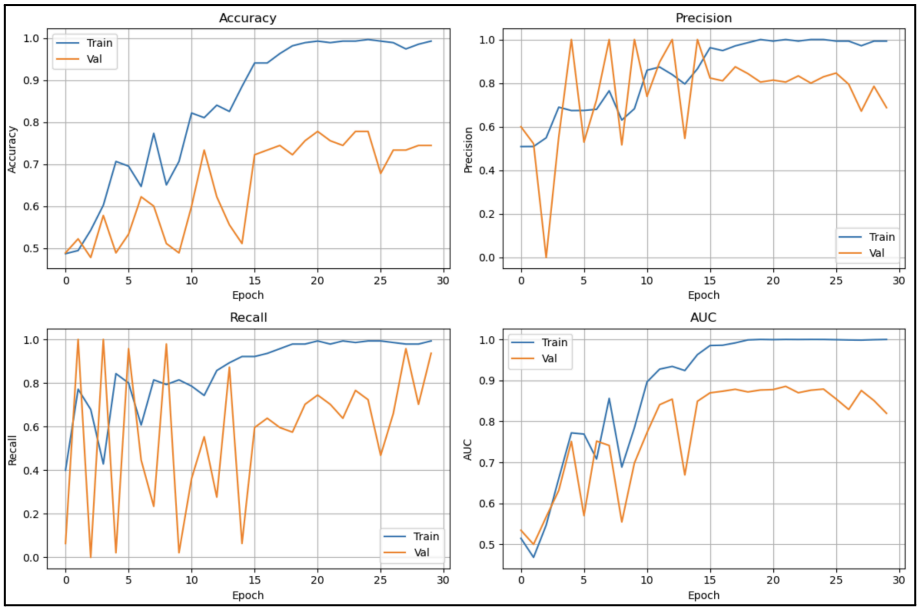


Figure 8: Plots - binary classifier II

Binary Model III Metrics:

Metric	Training Value	Validation Value
Accuracy	0.8569	0.7333
AUC	0.9622	0.8563
Loss	0.3129	0.5536
Precision	0.9486	0.7347
Recall	0.7788	0.7660

Table 4: Results for binary classifier III

The training metrics tell us that the model has trained not as well on the data with 85.69%, and an AUC close to 1.0. Keeping in mind the training metrics, the validation metrics are solid, a test accuracy of 73.3% and an AUC of 0.85. So overall, the gap between train and test is not too big as can be seen in the graphs.

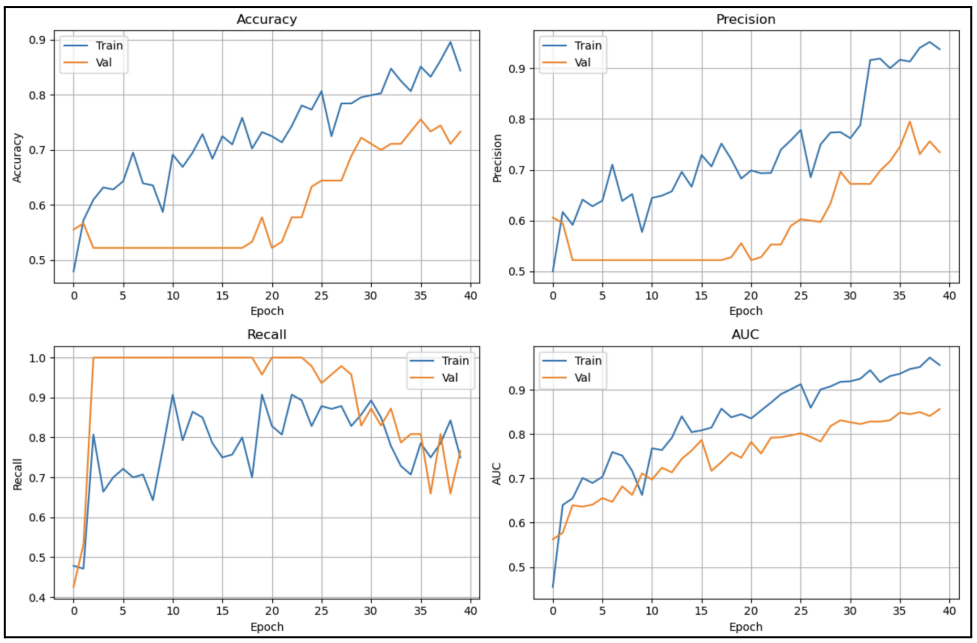


Figure 9: Plots - binary classifier III

Binary Model IV Metrics:

Metric	Training Value	Validation Value
Accuracy	0.9654	0.7444
AUC	0.9955	0.8199
Loss	0.2047	0.5149
Precision	0.9384	0.7727
Recall	0.9986	0.7234

Table 5: Results for binary classifier IV

This particular model has the same pattern as model I and II. The training metrics are pretty decent but they do not generalise well in the validation metrics. The graphs are highlighting the same thing, the training curves are smoother and upwards, but the validation curves have fluctuations.

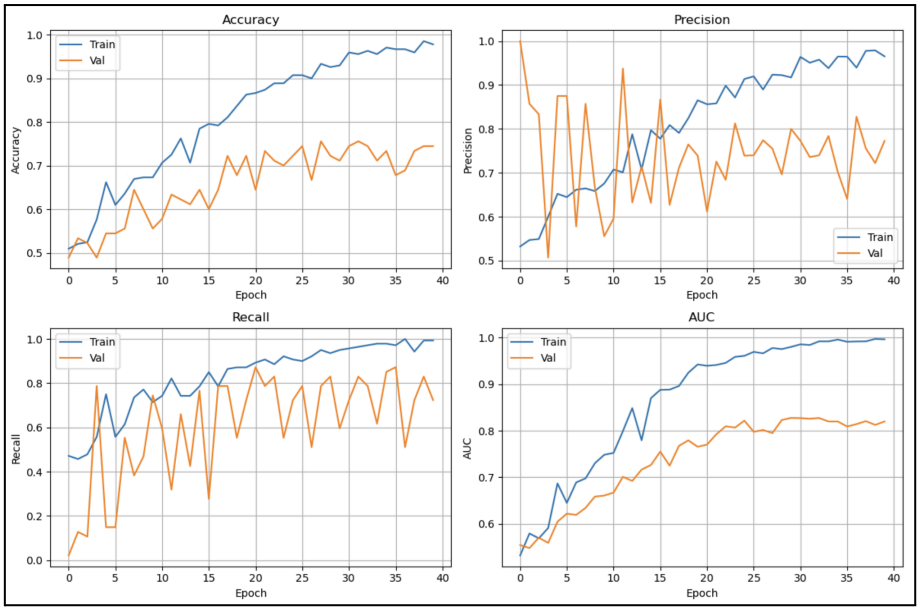


Figure 10: Plots - binary classifier IV

Confusion matrix for Binary Model I:

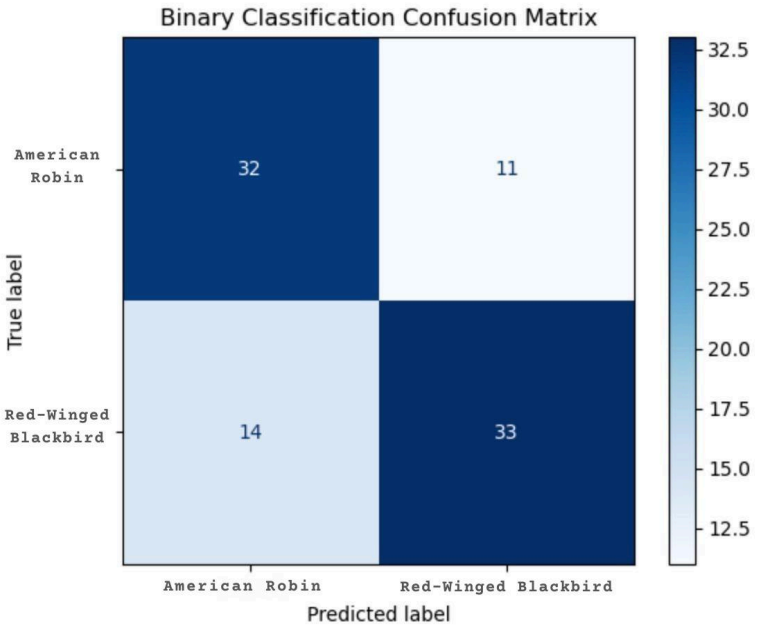


Figure 11: Confusion matrix - binary classifier I

The confusion matrix tells us that for American Robin 32 samples have been correctly classified and for Red-Winged Blackbird 33 samples have been correctly classified. 11 and 14 samples have been misclassified respectively. This is a pretty decent performance of the model.

Multi-Class Classifier I Metrics:

Metric	Training Value	Validation Value
Accuracy	0.6832	0.4504
Loss	1.0196	1.8292

Table 6: Results for multi-class classifier I

The subset model trains well, with an accuracy of 68.32%. But it does not generalise very well, and displays an accuracy of 45.04% only. It also takes significant time to train. The graph reiterates the observations in the table above.

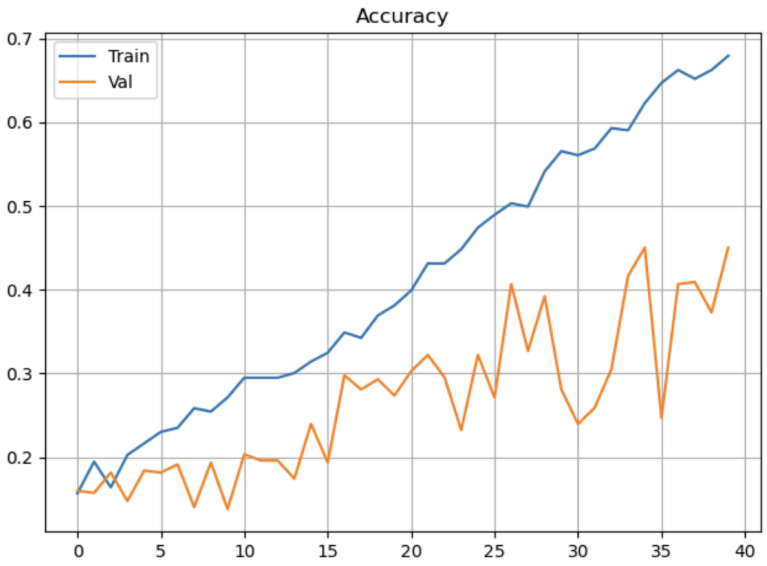


Figure 12: Plots – multi-class classifier I

Multi-Class Classifier II Metrics with Early Stopping:

Metric	Training Value	Validation Value
Accuracy	0.5304	0.4335
Loss	1.4665	1.7480

Table 7: Results for multi-class classifier II (early stopping)

The training value is only 53.04% and validation values is 43.35%. The difference between the train and test is not as much. So an okay model overall.



Figure 13: Plots – multi-class classifier II (early stopping)

Multi-Class Classifier III Metrics with Early Stopping:

Metric	Training Value	Validation Value
Accuracy	0.3393	0.0988
Loss	2.0636	3.3272

Table 8: Results for multi-class classifier II (early stopping)

Full dataset test accuracy is 13.31%. The training accuracy is 33.93% which is the lowest yet. This model neither trains well, neither does well on test data. The disparity is clearly displayed in the graph below.

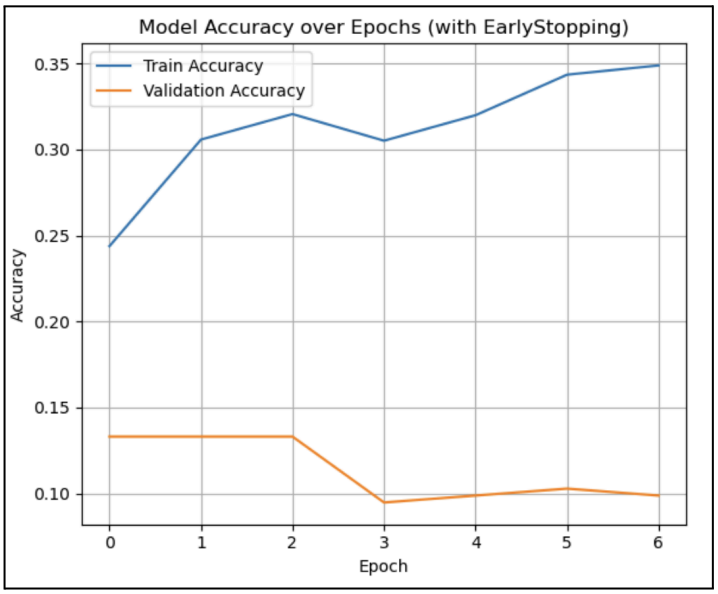


Figure 14: Plots – multi-class classifier II (early stopping)

Multi-Class Classifier IV Metrics:

Metric	Training Value	Validation Value
Accuracy	0.3129	0.2702
Loss	2.1269	2.2332

Table 9: Results for multi-class classifier II (early stopping)

Full dataset test accuracy is 33.06%. And the validation accuracy is 27.02%, which shows poor performance of the model. There is not much difference between train and test metrics so that's a silver lining, but this model neither trains well, nor geenralises well. The graph below re-iterates this point.

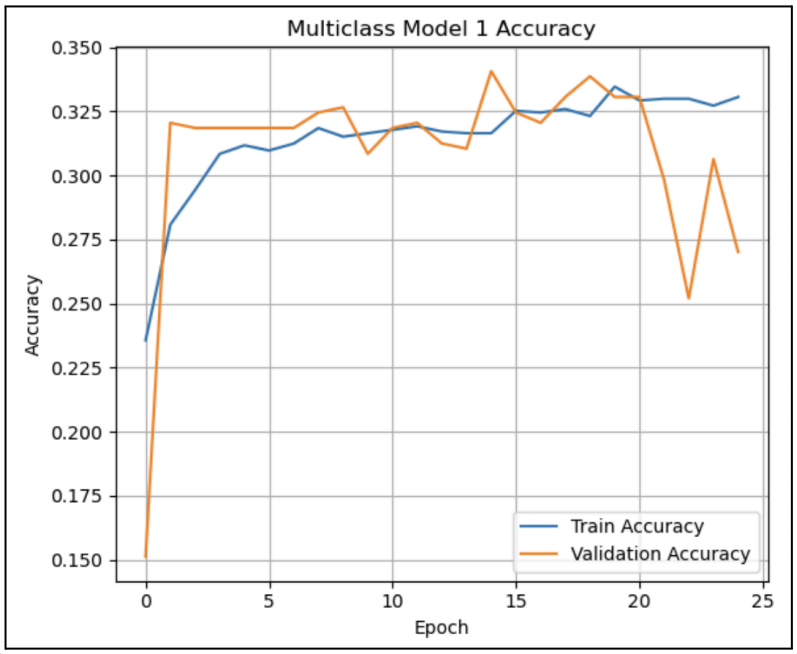


Figure 15: Plots – multi-class classifier II (early stopping)

Multi-Class Classifier V Metrics:

Metric	Training Value	Validation Value
Accuracy	0.3386	0.3448
Loss	2.1428	2.0742

Table 10: Results for multi-class classifier II (early stopping)

This model is a more complex version of the previous model. We dont see much difference in training metrics but we do see that this model generalises well, as there is a jump in the testing accuracy.

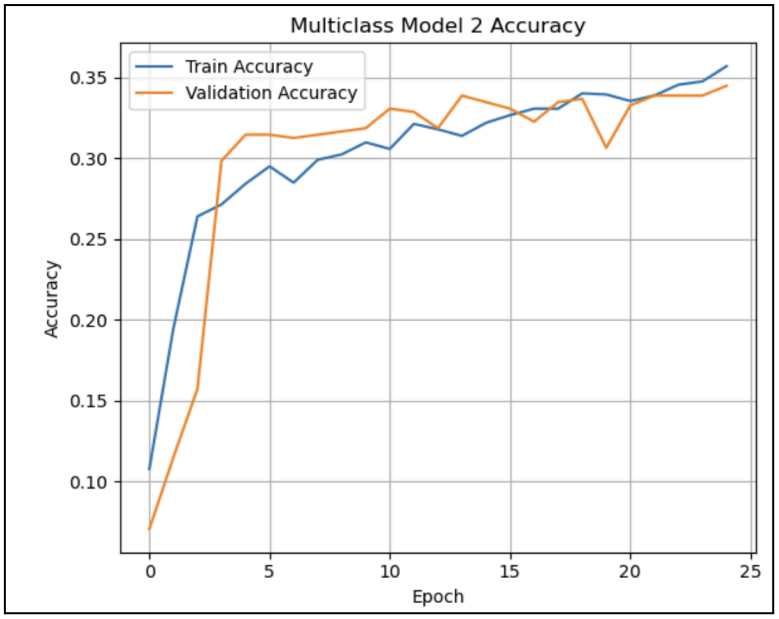


Figure 16: Plots – multi-class classifier II (early stopping)

Confusion Matrix for Multi-Class Classifier II:

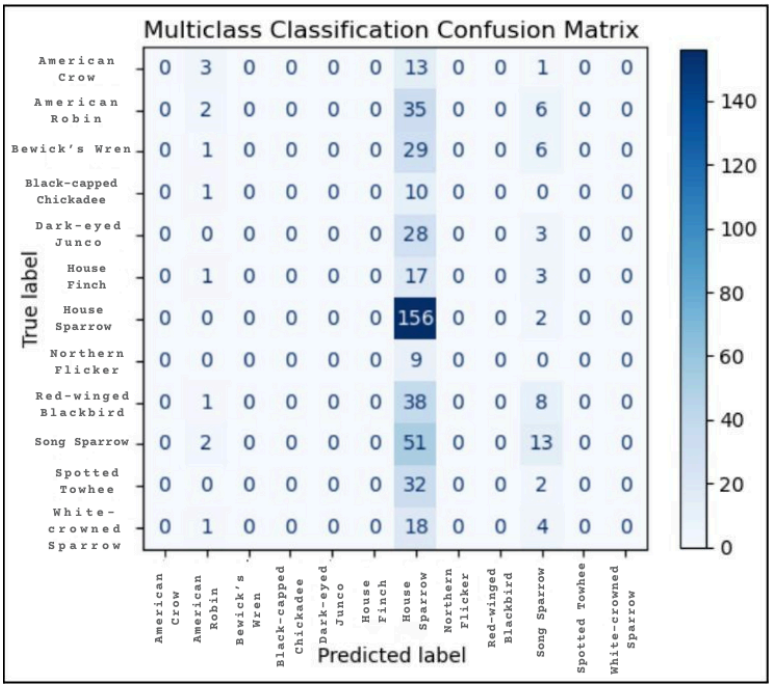


Figure 17: Confusion Matrix – multi-class classifier II

Above is the confusion matrix for the multi-class model II. We notice that species with more samples like House Sparrow and Song Sparrow perform well. But species who have lower amounts of samples in data get easily misclassified.

Discussion:

We also had three mp3 files as external test data to check the predictions of our models. multiple attempts were made but the model remained biased to one species or another. Though it is good at detecting multiple birds. Here are the findings:

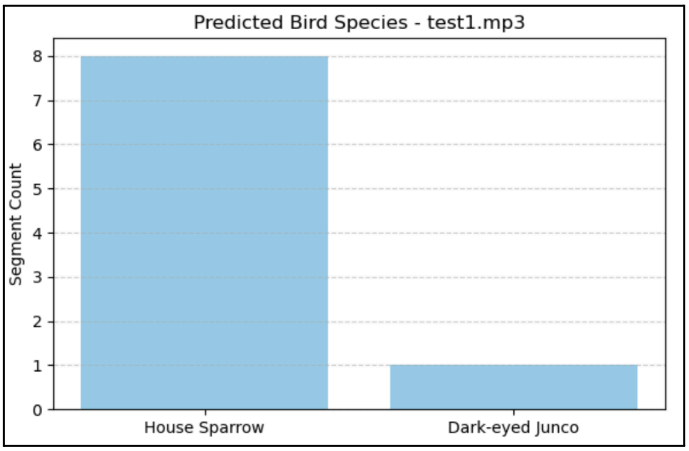


Figure 18: test1.mp3 results

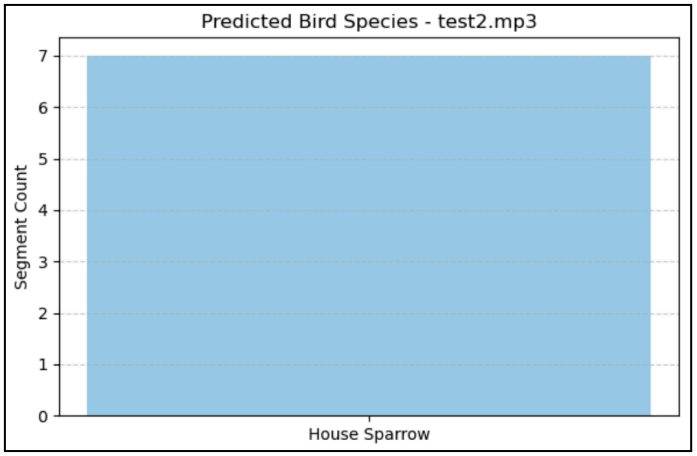


Figure 19: test2.mp3 results

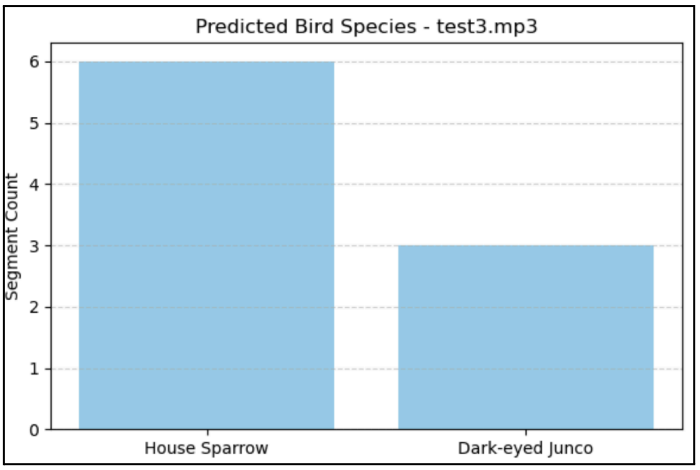


Figure 20: test3.mp3 results

As mentioned above, the model does seem biased toward the House Sparrow and the Dark-Eyed Junco. The graphs tell you the segment counts per species. The model does catch multiple bird sounds but whether or not it predicts the right birds is the question.

To test our theory about the model being biased toward some species, we plot the prediction distribution against the test set. And we see that some species like House Sparrow and American Robin have been predicted a lot of times. We see that the Dark-Eyed Junco and Song Sparrow are seventh and third on the graph, count wise. So there is a decent chance that the model may be right, but it is still biased with respect to some species. The confusion matrix above also re-iterates the same point.

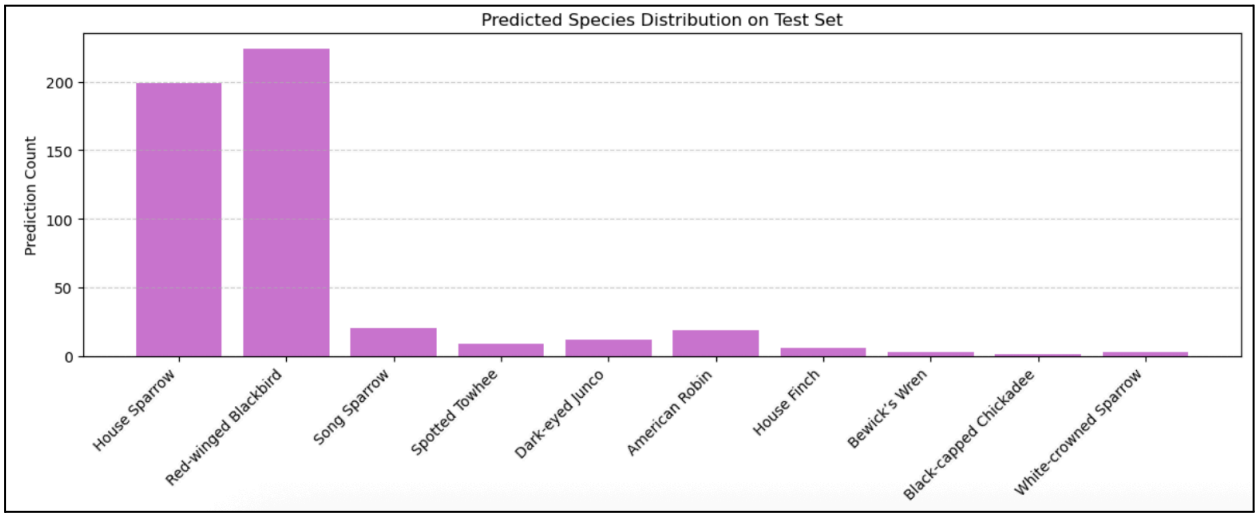


Figure 21: Plot - Species Distribution

The limitations for this project were mainly the hardware limitations, on a better machine, more epochs could be run, improving accuracy and other metrics. Another limitation was the class imbalance in some species, showcased by figure 5.
Time taken for binary classifiers was minimum but it took a good 25-35 minutes to train and model the multi-class classifiers.
House Finch, Bewick's Wren and White-Crowned Sparrow seem to have very low prediction counts. This could be due to their calls having a similar sound, or less samples in the dataset.
While testing external test data, Song Sparrow and Dark-eyed Junco appeared together, suggesting they have similar spectrograms.
We could have used RNNs here, because this is sequential data. But CNNs or neural networks in general make more sense because RNNs are slow and have short memory because their memory keeps getting refreshed.

Conclusion:

The models did pretty great on training data, sometimes even achieving 100% accuracy. But they did not fare too well on test data, since the validation accuracy did not cross 75% for binary and 35% for multi-class. The AUC for binary models was in the range of 0.75, which is decent. Some more improvements can certainly be brought to the multi-class models, as they have really poor validation accuracy.

For the external test data, the model did predict, but we cannot be too sure if the model is being biased or not. Our model may be biased towards some species as can be seen in the confusion matrix for multi-class, but it can detect whether there are multiple species or a single species in the audio files.

Improvements like more data samples per class or better class balancing will help. Also there was a limit to the number of epochs we could run due to hardware limitations.

Link to GitHub:
https://github.com/richa707/HW3

References:

IBM. (2021, October 6). *What is a neural network?* IBM. https://www.ibm.com/think/topics/neural-networks

Sankaran, S. (n.d.). *Understanding Convolutional Neural Networks (CNN)*. LearnOpenCV. Retrieved May 11, 2025, from https://learnopencv.com/understanding-convolutional-neural-networks-cnn/

Das, K. (2020, November 5). *How Recurrent Neural Network (RNN) Works*. Dataaspirant. Retrieved May 11, 2025, from https://dataaspirant.com/how-recurrent-neural-network-rnn-works/

Mendible, A. (2025). *Deep Learning* [Class notes]. DATA 5322, Seattle University.

Mendible, A. (2025). *Deep Learning II* [Class notes]. DATA 5322, Seattle University.